

Extended Technical Explanation

Traffic Correlation-Based Location Estimation in Anonymous Networks

File-by-File, Algorithm-by-Algorithm, Interview-Oriented Documentation

Document Purpose:	Deep technical understanding for interview, viva, and resume discussion
Primary Audience:	Student, faculty evaluators, interviewers, internship reviewers
Project Type:	Controlled cybersecurity research and full-stack demonstration platform
Main Technologies:	Python, Express.js, Next.js, Tailwind CSS, SQLite, WebSockets, TShark

Prepared By:	_____
Roll Number:	_____
Department:	_____
Institution:	_____
Guide:	_____
Academic Year:	_____

March 26, 2026

Contents

List of Tables

List of Figures

How to Use This Document

This document is intentionally much more detailed than the primary project report. The goal is not only to describe what the project does, but to help the student explain the project under pressure. If an interviewer asks about architecture, algorithms, database design, WebSockets, packet capture, file uploads, evaluation metrics, design tradeoffs, or future scope, the answers can be derived from the material here. The document is organized as a technical study guide.

The document has five main purposes. First, it explains the full system from the highest architectural level down to individual files. Second, it clarifies why each technology was chosen instead of other alternatives. Third, it documents the exact algorithmic pipeline from raw traffic metadata to probabilistic region estimation. Fourth, it highlights the boundaries of the project, especially the difference between controlled traffic analysis and real-world attribution claims. Fifth, it gives interview-oriented knowledge so that the project can be confidently discussed on a resume.

Chapter 1

Project Identity

1.1 One-Line Project Summary

The project is a controlled full-stack traffic-correlation platform that analyzes traffic metadata in simulation, replay, PCAP, and live-capture modes and estimates probable source regions through an explainable scoring and path-ranking pipeline.

1.2 Why This Project Exists

The project exists because anonymous-routing systems reduce direct traceability but do not remove all metadata. Even when payload contents remain encrypted, packet timing, packet size, sequence behavior, and burst patterns can still be analyzed. The project studies that problem in a controlled academic setting. It deliberately avoids claiming exact identity exposure, payload decryption, or operational surveillance. Instead, it turns metadata into structured events, extracts features, correlates candidate hops, ranks candidate paths, estimates likely origin regions, and presents the results through both a terminal and a full-stack web dashboard.

1.3 What Makes the Project Resume-Worthy

This project combines multiple engineering disciplines in a single coherent system. It includes Python systems programming, data modeling, correlation algorithms, API design, asynchronous job execution, WebSocket updates, authentication, SQLite persistence, file upload handling, frontend dashboard design, testing, and report writing. That breadth is valuable for internships because the project demonstrates both algorithmic thinking and product-level implementation.

1.4 Safe and Accurate Resume Framing

The safest and most accurate way to describe the project is:

Built a full-stack controlled traffic-correlation platform using Python, Express.js, Next.js, SQLite, and TShark to analyze simulated, replayed, PCAP, and live traffic metadata and estimate probable source regions in a cybersecurity research setting.

That wording is strong because it is technically accurate and ethically defensible.

Chapter 2

Scope, Promise, and Boundaries

2.1 What the System Does

The system performs controlled traffic analysis in four modes. In simulation mode, it creates synthetic Tor-like sessions with deterministic suspicious traffic patterns. In replay mode, it reads structured datasets from JSONL or CSV files. In PCAP mode, it uses TShark to convert packet captures into event-level traffic records and then into candidate sessions. In live mode, it captures packet metadata from a selected interface using TShark and processes it through the same PCAP-derived pipeline. Across all modes, the downstream analysis logic remains the same: feature extraction, hop correlation, path ranking, region estimation, evaluation, and result presentation.

2.2 What the System Does Not Do

The project does not decrypt encrypted payloads. It does not identify real individuals. It does not claim real-world deanonymization capability. It does not bypass security controls. It does not attempt to become an operational surveillance product. Those boundaries matter both ethically and technically. Real-world attribution is much harder, noisier, and more sensitive than controlled academic demonstration.

2.3 Why Those Boundaries Matter in an Interview

If an interviewer asks why the project is scoped this way, the correct answer is that meaningful engineering work can still be done without overclaiming capability. The platform demonstrates metadata correlation, interface design, back-end orchestration, and capture processing, but it avoids presenting uncertain inference as guaranteed truth. This is the correct research posture.

Chapter 3

End-to-End System Walkthrough

3.1 High-Level Pipeline

The core pipeline of the project can be written as:

Input Source → Raw Events → Feature Events → Hop Correlations → Ranked Paths → Region Estimation

The implementation is modular because each stage has a single responsibility. Inputs are acquired by the `capture` package. The `processing` package transforms and scores the traffic. The `service.py` module orchestrates the pipeline. The terminal renderer and frontend consume the structured output.

3.2 Request Flow in the Full Stack

When a user clicks a button on the frontend, the browser sends a request to the Express backend. The backend either creates a simulation session directly, accepts a dataset upload, or creates a live-capture request. Each request is stored in SQLite as a session and enqueued as a job. The queue invokes the Python engine by spawning `engine_api.py`. The Python process returns JSON on standard output. The backend stores that JSON result in the database and broadcasts a WebSocket message to connected clients. The frontend then refreshes the session list and the selected session details, allowing the user to inspect estimates, paths, correlations, and a terminal-style report.

3.3 Why This Architecture Is Good for Demonstration

This architecture separates concerns cleanly. The browser is not responsible for analysis logic. The backend is not responsible for implementing the traffic-correlation math directly. The Python engine is not responsible for UI rendering. Because of that separation, the system is easier to explain, easier to extend, and easier to test.

Chapter 4

Technology Stack: What Was Used and Why

4.1 Python

Python was chosen for the analysis engine because it is productive for data manipulation, event modeling, and algorithm development. The code is readable, fast enough for this project scale, and ideal for academic demonstration. Python also integrates well with command-line workflows, which makes it natural for both direct terminal usage and backend child-process execution.

4.2 Express.js

Express.js was chosen for the backend because it is lightweight, widely understood, and sufficient for API construction, upload handling, and authentication middleware. The project did not need a highly opinionated framework; it needed a pragmatic request router with middleware support. Express fits that requirement well.

4.3 Next.js

Next.js was chosen for the frontend because it provides a strong React-based application structure while remaining convenient to run locally. The app uses the App Router layout and a client-side dashboard page. The choice is appropriate for internship discussion because it shows familiarity with contemporary frontend patterns.

4.4 Tailwind CSS

Tailwind CSS was chosen for fast, consistent visual styling. The project does not use a heavy component library. Instead, it styles the dashboard through utility classes and a small custom CSS variable system in `globals.css`. This makes the interface visually distinct while keeping the style layer understandable.

4.5 SQLite

SQLite was selected because it is lightweight, file-based, and sufficient for local persistence. It avoids the operational complexity of running a separate database service while still demonstrating real persistence instead of in-memory-only session storage.

4.6 WebSockets

WebSockets were used because jobs are asynchronous. The frontend needs to know when queued jobs become running, completed, or failed without requiring constant manual refresh. WebSocket updates provide a clean solution for live progress and session refresh behavior.

4.7 TShark

TShark was selected because it provides robust packet metadata extraction from both PCAP files and live interfaces. Building a packet parser from scratch would have added substantial complexity and risk. TShark externalizes the low-level packet parsing problem and allows the project to focus on traffic-analysis logic.

4.8 `node:sqlite`

The backend uses the `node:sqlite` module, specifically `DatabaseSync`. This is an interesting technical decision because it keeps the backend dependency footprint small and uses built-in runtime capabilities available in modern Node environments. For an interview, that demonstrates awareness of evolving platform features.

Chapter 5

Repository Map

5.1 Top-Level Structure

The repository contains several categories of files:

- root Python and documentation files,
- capture and processing packages,
- backend source files,
- frontend source files,
- data and tests,
- logs and deployment artifacts.

5.2 Conceptual Directory Responsibilities

Path	Responsibility
capture/	Input acquisition and conversion from simulation, replay, PCAP, and live traffic
processing/	Feature extraction, scoring, path construction, estimation, and evaluation
backend/	API server, auth, persistence, job queue, Python engine execution, WebSockets
frontend/	Browser UI, controls, session list, tables, charts, and report rendering

Path	Responsibility
tests/	Automated validation of pipeline behavior and parser behavior
data/	Node registry and a legacy CSV artifact
logs/	Optional JSONL logs produced by the pipeline

Chapter 6

Core Data Model

6.1 Why the Data Model Matters

The project is easier to explain once the data model is understood. The entire pipeline transforms information through a series of progressively richer representations. The input begins as packet-like events or synthetic session steps. Those become feature-enriched records. Those feature records generate correlation candidates. Correlation candidates become ranked paths. Ranked paths support region estimates. Estimates and paths then support evaluation.

6.2 RawEvent

The `RawEvent` dataclass in `models.py` is the base observation unit used by the pipeline. It contains timestamp, packet size, direction, session ID, node ID, node type, source IP, destination IP, sequence number, region hint, and label. It also includes protocol, source and destination ports, flow ID, event span, and packet count.

Interview explanation: a `RawEvent` is not necessarily one raw network packet in every mode. In simulation and replay it usually corresponds to one logical path step. In PCAP and live mode it may represent an aggregated burst derived from one or more packets.

6.3 FeatureEvent

The `FeatureEvent` dataclass extends the meaning of a raw event by including derived properties such as `packet_delta` and `size_bucket`. That is important because correlation is usually more meaningful on transformed features than on absolute raw values.

6.4 CorrelationCandidate

The `CorrelationCandidate` dataclass captures a possible link between two adjacent stages, either ENTRY to MIDDLE or MIDDLE to EXIT. It stores raw deltas, normalized similarities, final score, session and label relationships, and node metadata.

6.5 RankedPath

The `RankedPath` dataclass represents a three-stage or partial path inferred from the correlation graph. It stores entry, middle, and exit node IDs; regions; path score; suspiciousness; whether sessions overlap; session ID; and whether the path is complete or partial.

6.6 RegionEstimate

The `RegionEstimate` dataclass is the final probabilistic output for a region. It stores the region label, normalized confidence score, support value, and number of supporting paths.

6.7 EvaluationSummary

The `EvaluationSummary` dataclass stores meta-level validation results such as suspicious session count, number of correctly estimated sessions, session accuracy, true origin region, and predicted origin region.

Chapter 7

Configuration Logic

7.1 Purpose of `config.py`

The `config.py` file centralizes thresholds, weights, priors, and logging paths. This is an important design decision because it avoids scattering hardcoded values across multiple files. Configuration-driven systems are easier to tune, easier to justify in a viva, and easier to extend.

7.2 Thresholds and Weights

The project uses:

- `time_threshold = 0.6,`
- `size_threshold = 160,`
- `time_weight = 0.45,`
- `size_weight = 0.35,`
- `sequence_weight = 0.20,`
- `score_threshold = 0.70.`

These values reflect a clear modeling assumption: time similarity is the most important factor, size similarity is the next most important factor, and sequence logic is a supportive constraint rather than the dominant signal.

7.3 Capture-Specific Relaxation

One of the more thoughtful parts of the design is that captured traffic uses relaxed thresholds. The configuration contains:

- `capture_time_threshold_multiplier`,
- `capture_size_threshold_multiplier`,
- `capture_score_threshold`.

This exists because simulation and replay are cleaner than real traffic. Real capture data needs looser matching to avoid collapsing to zero results.

7.4 Region Priors

The project includes simple region priors. In the current implementation they are neutral, but the architecture supports future non-uniform priors. This is useful in interviews because it shows that the estimator is written to support probabilistic weighting and not only raw score accumulation.

Chapter 8

Algorithmic Pipeline

8.1 Stage 1: Event Acquisition

The first stage depends on the mode. Simulation generates deterministic sessions. Replay loads structured rows from a file. PCAP runs TShark and parses fields. Live capture also runs TShark but on an interface and for a duration. The result in all cases is a list of `RawEvent` objects or a representation that is converted into those objects.

8.2 Stage 2: Feature Extraction

The `extract_features()` function in `processing/features.py` computes packet delta and size bucket and returns `FeatureEvent` instances. The packet delta is session-aware and computed in sorted session order. Size bucket compresses packet sizes into coarser classes, which can be useful for comparison and future extensions.

8.3 Stage 3: Hop Correlation

The `correlate_hops()` function in `processing/correlator.py` groups events by node type and compares candidate pairs across adjacent stages. It only compares ENTRY to MIDDLE and MIDDLE to EXIT. It enforces temporal ordering and rejects pairs that exceed effective thresholds. It then computes normalized similarities and a weighted score. Session and label bonuses adjust the score. Captured traffic may receive a protocol-based bonus if protocols match.

8.4 Stage 4: Path Construction

The `build_ranked_paths()` function in `processing/graph.py` joins compatible first-hop and second-hop correlations through a shared middle node. It produces a complete path only if session overlap logic is satisfied. If no complete path exists, it preserves partial paths from ENTRY to MIDDLE. This fallback is a pragmatic design choice made to avoid returning a totally empty result on sparse PCAP captures.

8.5 Stage 5: Region Estimation

The `estimate_regions()` function in `processing/estimator.py` sums path support by entry region, optionally multiplies suspicious paths by a bias factor, applies region priors, normalizes the totals, and returns a ranked list of `RegionEstimate` instances.

8.6 Stage 6: Evaluation

The `evaluate_estimates()` function in `processing/evaluator.py` derives a true suspicious origin region from suspicious entry events if such labels exist. It then computes how often ranked suspicious paths align with that region. This metric is strongest for simulation and replay, because they are labeled. It is less meaningful for unlabeled or weakly labeled live/PCAP scenarios.

8.7 Stage 7: Presentation

The terminal renderer in `ui/terminal.py` formats the summary into a readable plain-text report. The frontend renders the same underlying result as metrics, tables, charts, and a terminal-style text panel.

Chapter 9

Mathematics Behind the Scoring

9.1 Time Similarity

The time similarity formula is:

$$\text{time_similarity} = \frac{1}{1 + \frac{\Delta t}{T}}$$

This is a monotonically decreasing function. When the observed time gap is close to zero, the value approaches 1. As the time difference grows toward the threshold, the similarity decays smoothly. This is a practical choice because it avoids brittle step-function behavior.

9.2 Size Similarity

The size similarity formula is:

$$\text{size_similarity} = \frac{1}{1 + \frac{\Delta s}{S}}$$

It has the same interpretation as time similarity but is applied to packet-size difference. Small differences imply stronger similarity.

9.3 Sequence Similarity

The sequence similarity formula is:

$$\text{sequence_similarity} = \frac{1}{1 + \text{sequence_gap}}$$

In practice the implementation also constrains candidates to expected adjacent sequence numbers, making sequence more like a gate plus a supporting score.

9.4 Combined Score

The combined score is:

$$\text{score} = w_t \cdot \text{time_similarity} + w_s \cdot \text{size_similarity} + w_q \cdot \text{sequence_similarity} + \text{bonuses} - \text{penalties}$$

The system then clamps the result into the range $[0, 1]$. Clamping was added after earlier behavior allowed scores above 1.0, which made the output less realistic.

9.5 Why This Is Good for an Interview

This algorithm is easy to explain because it is transparent and interpretable. Unlike a black-box model, the candidate can explain exactly why a path received a high score: small timing gap, small size gap, correct stage ordering, matching session behavior, and possibly matching protocol in captured traffic.

Chapter 10

Simulation Pipeline

10.1 Purpose

Simulation is the cleanest mode. It generates deterministic Tor-like sessions with entry, middle, and exit nodes. Every fourth session is labeled suspicious and pushed toward a suspicious origin region.

10.2 Implementation Logic

The `generate_simulated_events()` function in `capture/simulate.py` uses a seeded random number generator. The deterministic seed makes the output reproducible. Suspicious sessions are biased to DE. Normal sessions are assigned other regions. Packet size and timing jitter are also different between suspicious and normal traffic, creating stronger correlation patterns for suspicious traffic.

10.3 Why It Matters

Simulation is useful because it validates the mathematical pipeline independent of noisy capture conditions. If the engine fails in simulation, the core logic is wrong. If the engine succeeds in simulation but struggles in PCAP, the issue lies in real-data conversion rather than the abstract correlation logic.

Chapter 11

Replay Pipeline

11.1 Purpose

Replay mode ensures deterministic file-based analysis. The dataset is not synthesized at runtime; it is loaded from structured files and passed into the same downstream logic as simulation and capture modes.

11.2 Implementation Logic

The `run_replay()` function in `capture/replay.py` decides whether to read JSONL or CSV. JSONL is handled by `read_jsonl()` and CSV by `read_csv_rows()`. Both return dictionaries that are converted into `RawEvent` objects.

11.3 Why It Matters

Replay mode demonstrates parser consistency and stable outputs. Because the dataset does not vary, it is a good mode for debugging UI or backend issues. If replay differs between runs, that indicates a software problem rather than environmental noise.

Chapter 12

PCAP Pipeline

12.1 Purpose

PCAP mode is the most realistic offline mode. It takes packet capture files, extracts metadata using TShark, converts packets into bursts, groups bursts into clusters, converts clusters into candidate sessions, and maps those sessions into stage-based events for the main pipeline.

12.2 Why PCAP Was the Hardest Part

Real PCAP data is noisy. Some packets are DNS, some are TCP ACKs, some are loopback traffic, some are very short-lived flows, and some do not align neatly with a three-stage route abstraction. Early versions of the parser over-compressed the data and returned only a few events. The parser was refined so that one rich cluster can now produce multiple candidate sequences instead of just one.

12.3 Important Heuristics

The PCAP parser uses several heuristics:

- flow grouping by ordered endpoint pair and protocol,
- burst splitting using idle gaps,
- burst clustering using time proximity,
- suspiciousness scoring on burst size, duration, and packet count,
- filtering of loopback-only traffic when more informative bursts exist,

- sequence chunking into groups of up to three bursts.

12.4 Why These Heuristics Are Acceptable Here

These heuristics are acceptable because this is a controlled academic project. They are explicit and explainable. The goal is not to present universal forensic truth. The goal is to show a working translation from messy packet metadata into structured correlation candidates.

Chapter 13

Live Capture Pipeline

13.1 Purpose

Live mode exists to make the system demonstrable in real time. It allows the user to click a button in the browser, choose an interface and duration, generate traffic during the capture window, and then inspect the resulting session.

13.2 Implementation Logic

The live pipeline is deliberately simple: `capture/live.py` calls the same TShark extraction helper used by PCAP and then calls the same row-to-event conversion logic. This is a strong design choice because it avoids implementing two different parsing systems for offline and live captured traffic.

13.3 Demonstration Advice

For a live demo, it is best to capture for a short period such as 8 seconds and generate traffic in another terminal using `ping` and `curl`. Long passive captures may produce sparse or uninteresting results.

Chapter 14

Backend Architecture

14.1 Express Server

The backend is started by `backend/server.js`. It creates the Express app, HTTP server, and WebSocket server; configures CORS and JSON parsing; creates an uploads directory; wires Multer storage; and exposes the API routes.

14.2 Authentication Flow

The backend exposes `/api/bootstrap` to reveal local demo credentials. The frontend uses that only for convenience. Real authentication occurs through `/api/auth/login`, which issues a random token stored in SQLite. In the strengthened version, tokens carry an expiry timestamp, can be revoked through `logout`, and are rejected automatically if expired. Protected routes are covered by `requireAuth`, and role-sensitive routes are further protected by `requireRole`.

14.3 Session Lifecycle

Every user-triggered action creates a session with an ID, mode, input label, status, config, and later a result or error. A corresponding job is queued in the `jobs` table. When the queue processes the job, the session changes from `queued` to `running` to `completed` or `failed`.

14.4 Why the Queue Exists

The queue exists because the backend should not block the request-response lifecycle waiting for Python analysis. Queueing also gives the UI a cleaner status model. This is especially important for live capture and larger uploads.

Chapter 15

Frontend Architecture

15.1 Single-Page Dashboard Design

The user experience is still centered on a single dashboard route, but the implementation is no longer a monolithic page file. The final frontend keeps the one-screen workflow for demonstration while splitting rendering and state responsibilities across reusable modules such as `frontend/components/dashboard/ui.js`, `frontend/hooks/useDashboard.js`, and `frontend/lib/dashboard-utils.js`. This preserves presentation simplicity without keeping all logic in one place.

15.2 State Management

State management is coordinated through the `useDashboard()` hook. That hook owns authentication state, session refresh logic, export actions, audit-log refresh behavior, login/logout flow, and role-aware UI actions, while the page file acts mainly as a composition layer. This is a stronger architecture than keeping all fetch and state logic inline inside a single component.

15.3 WebSocket Integration

The frontend opens a WebSocket connection after login. Whenever a session is created or updated, the socket handler refreshes the session list and the selected session. This makes the dashboard feel live without introducing a full state-management library.

Chapter 16

Why the Project Has Both CLI and Web UI

The CLI remains useful because it is the simplest path to validate the engine. It gives deterministic text output and is useful during debugging, testing, and academic explanation. The web UI makes the project easier to present, easier to inspect, and more suitable for a non-terminal audience. Keeping both interfaces is a sign of good engineering modularity because both depend on the same service layer instead of duplicating logic.

Chapter 17

File-by-File Explanation

17.1 How to Read This Chapter

The following chapters explain every meaningful file in the repository. For each file, the explanation covers purpose, why it exists, what problem it solves, what important functions it contains, how it interacts with other files, and how to explain it in an interview.

Chapter 18

Root-Level Files

18.1 `README.md`

Purpose. The README is the public-facing entry point to the repository. It summarizes the project, scope, stack, setup steps, API endpoints, and demo flow.

Why it exists. A good README lowers the onboarding cost for reviewers, faculty, and recruiters. It tells someone what the project is before they dive into the code.

Interview explanation. If asked why README quality matters, the answer is that good engineering includes communication and documentation, not only source code.

18.2 `main.py`

Purpose. This is the CLI entry point for human-readable terminal output.

What it does. It parses arguments such as mode, dataset, sessions, seed, interface, capture duration, and top-k setting. It then calls `run_pipeline()` and prints the report.

Why it exists. It provides a clean command-line interface independent of the web application.

Important design point. The CLI does not reimplement the pipeline. It delegates to `service.py`. This is good separation of concerns.

18.3 `engine_api.py`

Purpose. This is the structured JSON interface for the Python engine.

What it does. It parses command-line arguments like `main.py`, but instead of printing a terminal report, it prints machine-readable JSON.

Why it exists. The backend needs a structured contract with Python. Parsing plain text would be fragile. JSON is the correct choice.

Interview explanation. This file is the bridge between a Python analytics engine and a Node backend.

18.4 `service.py`

Purpose. This is the orchestration layer of the analysis engine.

What it does. It selects the input source based on mode, runs feature extraction, correlation, path ranking, estimation, evaluation, optional logging, protocol summarization, and final report rendering. It returns a structured dictionary containing meta, summary, estimates, evaluation, paths, correlations, sampled raw events, protocol summaries, and the terminal report.

Why it exists. Without this file, the project would duplicate workflow logic between the CLI and API bridge.

18.5 `models.py`

Purpose. Defines all core data structures used in the pipeline.

Why it matters. The project becomes much easier to reason about because each pipeline stage transforms a specific dataclass rather than passing around loose dictionaries everywhere.

18.6 `config.py`

Purpose. Centralized configuration for thresholds, weights, priors, and log locations.

Why it matters. It makes tuning and reasoning about the algorithm possible.

18.7 `requirements.txt`

Purpose. Placeholder for Python dependencies. In the current repository it is empty.

Why it is still notable. Even empty files can indicate intended package-management structure. In future, dependencies like `scapy` or analysis utilities may be listed here.

18.8 `package.json`

Purpose. Root script runner for convenience commands.

What it contains. Scripts for backend dev, frontend dev, and Python tests.

18.9 `docker-compose.yml`

Purpose. Container orchestration for frontend and backend.

Why it matters. It shows that the project was designed with deployment and reproducibility in mind, even though live capture remains a host-sensitive feature. In the upgraded version it also captures environment defaults for multiple user roles and persistent mounts for backend data and uploads.

18.10 `main.tex`

Purpose. The academic report.

Why it matters. It shows the project is not only code-complete but also documentation-ready for institutional submission.

18.11 `explanation.tex`

Purpose. This document.

Why it matters. It is a deep technical companion for interviews, resume discussion, and exhaustive understanding.

18.12 `analyze.py`

Purpose. A legacy exploratory script that loads `data/network_features.csv` using Pandas, prints basic statistics, and plots protocol distribution.

Why it exists. It likely reflects an early experimentation phase before the full modular platform was built.

Interview explanation. This file demonstrates project evolution: the student started with exploratory analysis and later built a far more structured system.

18.13 `sniffer.py`

Purpose. A legacy packet sniffer based on Scapy.

What it does. Captures packets, extracts a few basic features, and saves them to CSV.

Why it matters. Like `analyze.py`, it shows the project's prototype lineage. It is not the main production path now, but it documents the earlier approach.

Chapter 19

Capture Package

19.1 `capture/__init__.py`

Purpose. Package marker.

19.2 `capture/simulate.py`

Purpose. Synthetic event generation.

Important functions. `generate_simulated_events()` and `_pick_path()`.

Why it matters. It provides deterministic and explainable synthetic data for algorithm validation.

19.3 `capture/replay.py`

Purpose. Replay loader for JSONL and CSV files.

Important function. `run_replay()`.

Why it matters. It ensures the same engine can operate on file-based historical data.

19.4 `capture/pcap.py`

Purpose. PCAP parsing and conversion into `RawEvent` instances.

Important pieces. `PacketRecord`, `FlowBurst`, `load_pcap_events()`,
`rows_to_events()`, `_extract_tshark_rows()`, `_rows_to_packets()`,
`_packets_to_events()`, `_build_flow_bursts()`, `_cluster_bursts()`,

`_cluster_to_sequences()`, and helper functions for mapping, classification, flow keys, and protocol fingerprinting.

Why it matters. This file is the bridge from low-level captured packets to the higher-level path abstraction used by the core algorithm. It is also the most heuristic-heavy file in the repository. The upgraded parser distinguishes metadata patterns such as TLS 1.3, TLS 1.2, HTTP/2, QUIC, DNS over HTTPS, and DNS over TLS when the underlying capture fields support those inferences.

Interview explanation. This is the best file to discuss when asked about turning raw system data into structured algorithmic inputs.

19.5 `capture/live.py`

Purpose. Live capture orchestration using TShark.

Why it matters. It reuses the PCAP parsing logic rather than creating a separate interpretation path.

Chapter 20

Processing Package

20.1 `processing/__init__.py`

Purpose. Package marker.

20.2 `processing/features.py`

Purpose. Feature derivation layer.

Why it matters. It moves the system from raw observations to comparison-ready records.

20.3 `processing/correlator.py`

Purpose. Candidate scoring engine.

Why it matters. This is the core mathematical heart of the system.

20.4 `processing/graph.py`

Purpose. Path construction from hop-level candidates.

Why it matters. It transforms pairwise edges into route-level objects, which are easier to reason about and estimate from.

20.5 `processing/estimator.py`

Purpose. Region support aggregation and normalization.

Why it matters. It converts path-level evidence into the final user-facing estimate.

20.6 `processing/evaluator.py`

Purpose. Validation summary.

Why it matters. It provides a disciplined way to discuss how good the output is, especially on labeled data.

Chapter 21

Data Package and Artifacts

21.1 `data/__init__.py`

Purpose. Package marker.

21.2 `data/nodes.py`

Purpose. Registry of known entry, middle, and exit nodes plus regions and IPs.

Why it matters. Simulation and PCAP mapping need a stable node universe to produce path-like outputs.

21.3 `data/network_features.csv`

Purpose. Legacy data artifact used by `analyze.py`.

Why it matters. It records the earlier exploratory phase of the project.

Chapter 22

UI Package

22.1 `ui/__init__.py`

Purpose. Package marker.

22.2 `ui/terminal.py`

Purpose. Human-readable terminal report renderer.

Why it matters. It provides a compact engineering-friendly view of the pipeline output and is reused conceptually by the dashboard's terminal report area.

Chapter 23

Utility Files

23.1 `utils/__init__.py`

Purpose. Package marker.

23.2 `utils/io.py`

Purpose. File IO helpers for JSONL and CSV.

Why it matters. Centralizing read and write logic keeps the capture modules smaller and easier to test.

Chapter 24

Backend Files

24.1 `backend/package.json`

Purpose. Backend package manifest.

Dependencies. `express`, `cors`, `multer`, `ws`, and `pdfkit`.

24.2 `backend/server.js`

Purpose. Main backend entry point.

Why it matters. This is the server process that creates the HTTP API and WebSocket server.

Updated responsibilities. In the upgraded version it also exposes CSV, HTML, and server-side PDF export endpoints, plus admin-only user-management routes for listing, creating, and updating users.

24.3 `backend/auth.js`

Purpose. Authentication helper layer.

Why it matters. It isolates token extraction, login, route protection, and role checks from route definitions.

Updated responsibilities. The upgraded version also performs expired-token cleanup and exposes logout logic that revokes tokens instead of relying only on client-side state clearing.

24.4 backend/db.js

Purpose. SQLite schema initialization and data access layer.

Why it matters. It provides real persistence and a clear session model.

Updated responsibilities. The database layer now seeds administrative, analyst, and viewer demo accounts; stores token expiry and revocation timestamps; captures audit logs; and exposes helper methods for listing users, creating users, and updating roles or passwords.

24.5 backend/queue.js

Purpose. Queued background execution of sessions.

Why it matters. It decouples request acceptance from long-running analysis.

24.6 backend/pythonRunner.js

Purpose. Launches the Python engine and parses JSON output.

Why it matters. This file is the cross-language bridge.

24.7 backend/socketHub.js

Purpose. Centralized WebSocket upgrade and broadcast logic.

Why it matters. It prevents WebSocket concerns from polluting route and queue code.

24.8 backend/Dockerfile

Purpose. Backend containerization instructions.

Why it matters. The final Dockerfile installs Node.js runtime dependencies, Python 3, and TShark so that the container has the same cross-language execution surface as the host-based development workflow.

24.9 backend/.env.example

Purpose. Example backend configuration for demo credentials and runtime settings.

Chapter 25

Frontend Files

25.1 `frontend/package.json`

Purpose. Frontend package manifest.

Updated dependencies. The final version adds `reactflow` so that topology visualization is implemented with a dedicated graph library rather than a custom SVG-only rendering approach.

25.2 `frontend/app/layout.js`

Purpose. Root layout wrapper and metadata declaration.

25.3 `frontend/app/globals.css`

Purpose. Global color palette, base styling, panel styling, and page background design.

Updated responsibilities. The stylesheet now also defines the dashboard animation system, topology node styling, graph-panel presentation, and accessibility-minded reduced-motion behavior.

25.4 `frontend/app/page.js`

Purpose. Entire dashboard UI.

Why it matters. In the refactored version this file is now intentionally thinner and delegates most UI rendering and state management to reusable modules.

Updated responsibilities. In the upgraded version this file also contains the React Flow topology panel, CSV/PDF export controls, protocol summary rendering, role-aware interface restrictions, and the admin user-management panel.

Interview explanation. This file demonstrates frontend state orchestration, data fetching, result presentation, graph-based visualization, and role-aware product behavior in a single coherent page.

25.5 `frontend/components/dashboard/ui.js`

Purpose. Reusable dashboard presentation layer.

Why it matters. This file extracts the large collection of presentational components from the old monolithic page file. It now contains session cards, tables, charts, topology visualization, authentication screen, export panel, audit panel, and admin user-management UI.

25.6 `frontend/hooks/useDashboard.js`

Purpose. Centralized client-side state and action management for the dashboard.

Why it matters. This refactor reduces page-level complexity by collecting fetch logic, session refresh logic, login/logout actions, audit refresh logic, and export handlers into a single hook that can be tested and reasoned about independently from the JSX layout.

25.7 `frontend/jsconfig.json`

Purpose. Frontend project configuration.

25.8 `frontend/next.config.mjs`

Purpose. Next.js runtime configuration.

25.9 `frontend/postcss.config.mjs`

Purpose. Tailwind/PostCSS integration.

25.10 `frontend/Dockerfile`

Purpose. Frontend containerization instructions.

Why it matters. The final version runs the Next.js development server on 0.0.0.0, which is important for container accessibility from the host browser.

25.11 `frontend/.env.example`

Purpose. Example frontend environment configuration, primarily the API base URL.

Chapter 26

Tests and Sample Data

26.1 `tests/test_pipeline.py`

Purpose. Validates the simulation and replay pipeline.

Why it matters. It checks that the engine still produces sensible estimates and that CSV and JSONL replay match.

26.2 `tests/test_pcap_parser.py`

Purpose. Validates that PCAP row parsing produces clustered events with sequence information and flow IDs.

26.3 `tests/sample_data/replay_dataset.jsonl`

Purpose. Structured replay fixture in JSONL format.

26.4 `tests/sample_data/replay_dataset.csv`

Purpose. Structured replay fixture in CSV format.

26.5 `tests/sample_data/live_capture.pcap`

Purpose. Earlier PCAP sample, useful mainly as a historical/debug fixture.

26.6 tests/sample_data/better_capture.pcap

Purpose. Intermediate PCAP sample used during parser refinement.

26.7 tests/sample_data/better_capture2.pcap

Purpose. Stronger PCAP sample that currently demonstrates the refined parser effectively.

Chapter 27

Logs and Outputs

27.1 `logs/raw_events.jsonl`

Optional log of raw events.

27.2 `logs/features.jsonl`

Optional log of feature events.

27.3 `logs/correlations.jsonl`

Optional log of correlation candidates.

27.4 `logs/estimates.jsonl`

Optional log of region estimates.

27.5 Why Logging Matters

Logging matters because it makes the pipeline inspectable stage by stage. If an interviewer asks how you would debug wrong estimates, the correct answer is to inspect logs progressively: raw events, features, correlations, and final estimates.

Chapter 28

Deployment and Hosting Discussion

28.1 Local Deployment

The current project is best run locally for full functionality because live capture and TShark require host-level access.

28.2 Hosted Deployment

The frontend can be deployed on Vercel, but the backend is better suited to Render, Railway, or a VPS because it uses Express, WebSockets, SQLite, Python child processes, and optional TShark integration.

28.3 Why Vercel Alone Is Not Enough

Vercel excels at frontend and serverless scenarios, but this backend expects a long-running server process and local tooling. That mismatch should be explained clearly if asked about deployment decisions.

Chapter 29

Strengths of the Project

The project has several strong engineering qualities. It is modular. It has a clean service boundary between Python and Node. It supports multiple input modes through a shared downstream pipeline. It includes persistence rather than using only in-memory state. It offers both terminal and browser output. It uses explicit algorithms rather than opaque black boxes. It handles asynchronous workflow through queued jobs and WebSocket updates. It has tests. It has documentation. It demonstrates iterative refinement, especially in the PCAP parser.

Chapter 30

Weaknesses and Limitations

The project also has real limitations. PCAP and live inference remain heuristic-heavy. Evaluation metrics are strongest on labeled data, even though the benchmark suite is now broader than the initial single-run checks. SQLite has been tuned with WAL mode, foreign-key enforcement, busy timeouts, and persisted job recovery, but it is still not the best long-term choice for multi-instance hosted deployments. The queue now supports retry and stale-job recovery, yet it remains an in-process worker rather than a distributed queue. The legacy scripts also show that the repository contains exploratory artifacts from the project's earlier phase.

Chapter 31

What Could Be Improved Next

31.1 Algorithmic Improvements

Possible improvements include richer protocol modeling, better flow segmentation, support for windowed multi-burst matching, graph-based confidence visualization, and formal benchmarking against larger labeled datasets.

31.2 Backend Improvements

Possible backend improvements include stronger password policy enforcement, better upload validation, a persistent external queue such as Redis-backed workers, and a full migration from SQLite to PostgreSQL using the prepared schema reference.

31.3 Frontend Improvements

The frontend refactor has already been completed at the component and hook level, so the next frontend improvements are more about polish than structure. Useful next steps include richer charts, better filtering and sorting, denser comparison views, and more advanced graph interaction for larger session sets.

31.4 DevOps Improvements

The project could gain CI pipelines, containerized integration tests, deployment profiles, and better environment separation for dev, demo, and hosted modes.

Chapter 32

Interview Questions and Strong Answers

32.1 What problem does this project solve?

It solves the problem of turning traffic metadata into structured, explainable correlation and region-estimation results across simulation, replay, PCAP, and live modes in a controlled cybersecurity research setting.

32.2 Why did you choose Python for the engine?

Because the engine is algorithm-heavy and benefits from concise data modeling, readable logic, and strong CLI ergonomics. Python was a practical choice for rapid iteration and clear academic explanation.

32.3 Why not keep everything in Node.js?

Separating the analysis engine into Python made the core scoring logic easier to experiment with and easier to validate independently from the web stack. The backend and frontend then consume it as a service boundary.

32.4 Why did you use WebSockets?

Because jobs are asynchronous. WebSockets let the frontend react to job state changes without polling aggressively.

32.5 Why SQLite?

It provides real persistence with minimal operational overhead. For a controlled academic tool, it is a good tradeoff.

32.6 What was the hardest part?

The hardest part was PCAP parsing. Real packet captures are noisy and do not naturally align with a simple three-hop abstract model. The parser had to be refined so it preserved enough meaningful burst structure for the downstream engine to produce non-empty ranked paths.

32.7 What is the biggest technical tradeoff in the project?

The biggest tradeoff is between realism and explainability. Highly realistic traffic-analysis systems are complex and uncertain. This project intentionally uses explicit heuristics and transparent formulas so it remains explainable and defensible.

32.8 What would you improve if given more time?

I would improve protocol-specific flow modeling, move the persisted queue to an external worker system, migrate hosted deployments to PostgreSQL, strengthen password policy and operational hardening, and build a richer benchmarking suite for more diverse capture datasets.

Chapter 33

How to Explain the Project in Two Minutes

This project is a controlled full-stack traffic-correlation platform designed for cybersecurity research and academic demonstration. It accepts four input modes: simulation, replay, PCAP, and live capture. The Python engine converts those inputs into raw events, extracts features, scores adjacent hops using timing, size, and sequence similarity, builds ranked paths, and estimates probable origin regions. The Express backend manages authentication, persistence, uploads, queued jobs, and WebSocket updates. The Next.js frontend provides a dashboard for launching sessions and inspecting the results visually. The project is explainable, modular, and presentation-ready, while staying inside ethical research boundaries.

Chapter 34

How to Explain the Project in Deep Technical Detail

At the data-model level, the system transforms observations through dataclasses: `RawEvent`, `FeatureEvent`, `CorrelationCandidate`, `RankedPath`, `RegionEstimate`, and `EvaluationSummary`. At the capture level, simulation and replay produce clean event streams, while PCAP and live modes use TShark to extract metadata and convert packets into burst-level structures. At the processing level, the engine computes temporal and size-based similarity under configurable thresholds, applies bonuses and penalties, filters weak candidates, and then joins compatible hop pairs through a graph-building stage. At the application level, the backend stores jobs and sessions in SQLite and invokes the Python engine as a child process. At the presentation level, the frontend receives updates over WebSockets and renders both structured tables and a terminal-like report.

Chapter 35

Detailed Setup Notes

35.1 Backend Environment

Create `backend/.env` from `backend/.env.example`. Typical values are:

```
APP_DEFAULT_USER=admin
APP_DEFAULT_PASS=admin123
APP_ANALYST_USER=analyst
APP_ANALYST_PASS=analyst123
APP_VIEWER_USER=viewer
APP_VIEWER_PASS=viewer123
HOST=0.0.0.0
PORT=4000
AUTH_TOKEN_TTL_HOURS=8
```

35.2 Frontend Environment

Create `frontend/.env.local` from `frontend/.env.example`. Typical values are:

```
NEXT_PUBLIC_API_BASE=http://localhost:4000
```

35.3 Install and Run

```
npm --prefix backend install
npm --prefix frontend install
python3 -m unittest discover -s tests -v
npm run benchmark:python
npm --prefix backend run dev
```

```
npm --prefix frontend run dev
```

35.4 Hosted Caveat

The hosted version should favor simulation, replay, and possibly PCAP upload. Real live capture should remain local because hosted environments do not provide the same packet-capture privileges as a developer machine.

Chapter 36

Suggested Viva and Resume Language

36.1 Viva Summary

This project demonstrates a controlled traffic-analysis platform where metadata from simulated, replayed, offline packet captures, and live network observation is transformed into correlation candidates, ranked routing paths, and probabilistic source-region estimates. The platform is implemented as a modular Python engine with a Node backend and a React-based dashboard that includes a dedicated topology graph, protocol fingerprint summaries, exportable reports, and role-based analyst workflows.

36.2 Resume Bullet 1

Developed a Python-based traffic-correlation engine with feature extraction, hop scoring, path ranking, probabilistic region estimation, protocol fingerprinting, and evaluation across simulated, replayed, PCAP, and live-capture inputs.

36.3 Resume Bullet 2

Built a full-stack dashboard using Express.js, SQLite, WebSockets, Next.js, Tailwind CSS, and React Flow for authenticated session management, role-aware access control, token expiry/logout handling, dataset uploads, queued analysis jobs, and real-time topology visualization.

36.4 Resume Bullet 3

Integrated TShark-based packet metadata ingestion and refined PCAP burst clustering logic to convert real packet captures into structured analysis sessions, then added analyst-facing CSV/PDF export, audit logging, and admin user-management capabilities for presentation-ready demonstrations.

Chapter 37

Final Assessment

This project is sufficiently broad and technically rich to discuss in an interview or internship setting. It demonstrates system design, cross-language integration, algorithmic modeling, asynchronous orchestration, UI engineering, graph-based visualization, server-side document generation, access-control design, audit logging, testing, and documentation. Its strengths should be presented confidently, and its limitations should be presented honestly. That combination usually leaves a stronger impression than overclaiming.

Appendix A: Quick Command Reference

```
python3 main.py --mode simulate --sessions 16 --top-k 6
python3 main.py --mode replay --dataset tests/sample_data/replay_dataset.jsonl
python3 main.py --mode replay --dataset tests/sample_data/replay_dataset.csv
python3 main.py --mode pcap --dataset tests/sample_data/better_capture2.pcap
python3 main.py --mode live --interface any --capture-seconds 8
python3 engine_api.py --mode simulate --sessions 18 --top-k 8
python3 -m unittest discover -s tests -v
npm run benchmark:python
npm --prefix backend run dev
npm --prefix frontend run dev
docker-compose up --build
```

Appendix B: File Inventory

File	Role in Repository
README.md	Top-level project overview
main.py	Terminal CLI entry point
engine_api.py	JSON bridge for backend integration
service.py	Shared pipeline orchestrator
models.py	Core dataclasses
config.py	Central thresholds and weights
analyze.py	Legacy exploratory analysis script
sniffer.py	Legacy Scapy-based packet capture prototype
capture/simulate.py	Synthetic session generator
capture/replay.py	Replay loader
capture/pcap.py	PCAP parsing and burst logic
capture/live.py	Live capture wrapper
processing/features.py	Feature extraction
processing/correlator.py	Similarity scoring
processing/graph.py	Ranked path construction
processing/estimator.py	Region estimator
processing/evaluator.py	Evaluation summary builder
ui/terminal.py	Terminal report renderer
utils/io.py	JSONL and CSV helpers
data/nodes.py	Node registry and region metadata
backend/server.js	Express server and routes
backend/auth.js	Auth helper layer
backend/db.js	SQLite initialization, auth token storage, and audit-log data access
backend/queue.js	Job queue logic
backend/pythonRunner.js	Python process bridge
backend/socketHub.js	WebSocket broadcast support
backend/postgres_schema.sql	PostgreSQL-ready schema reference

File	Role in Repository
<code>docker-compose.yml</code>	Container orchestration for frontend/backend demo stack
<code>frontend/app/page.js</code>	Thin dashboard page entry point
<code>frontend/components/dashboard/index.js</code>	Dashboard presentation components
<code>frontend/hooks/useDashboard.js</code>	Dashboard state and action hook
<code>frontend/app/layout.js</code>	Root layout
<code>frontend/app/globals.css</code>	Global styling
<code>tests/test_pipeline.py</code>	Pipeline validation
<code>tests/test_pcap_parser.py</code>	Parser validation
<code>tests/test_benchmark_regression.py</code>	Regression benchmark regression test
<code>tests/benchmark_pipeline.py</code>	Repeatable benchmark summary generator

Appendix C: References and Knowledge Sources

Bibliography

- [1] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The Second-Generation Onion Router,” in *Proceedings of the 13th USENIX Security Symposium*, 2004.
- [2] S. J. Murdoch and G. Danezis, “Low-Cost Traffic Analysis of Tor,” in *IEEE Symposium on Security and Privacy*, 2005.
- [3] Wireshark Foundation, “Wireshark and TShark Documentation.” Available: <https://www.wireshark.org/docs/>
- [4] OpenJS Foundation, “Express.js Documentation.” Available: <https://expressjs.com/>
- [5] Vercel, “Next.js Documentation.” Available: <https://nextjs.org/docs>
- [6] Tailwind Labs, “Tailwind CSS Documentation.” Available: <https://tailwindcss.com/docs>